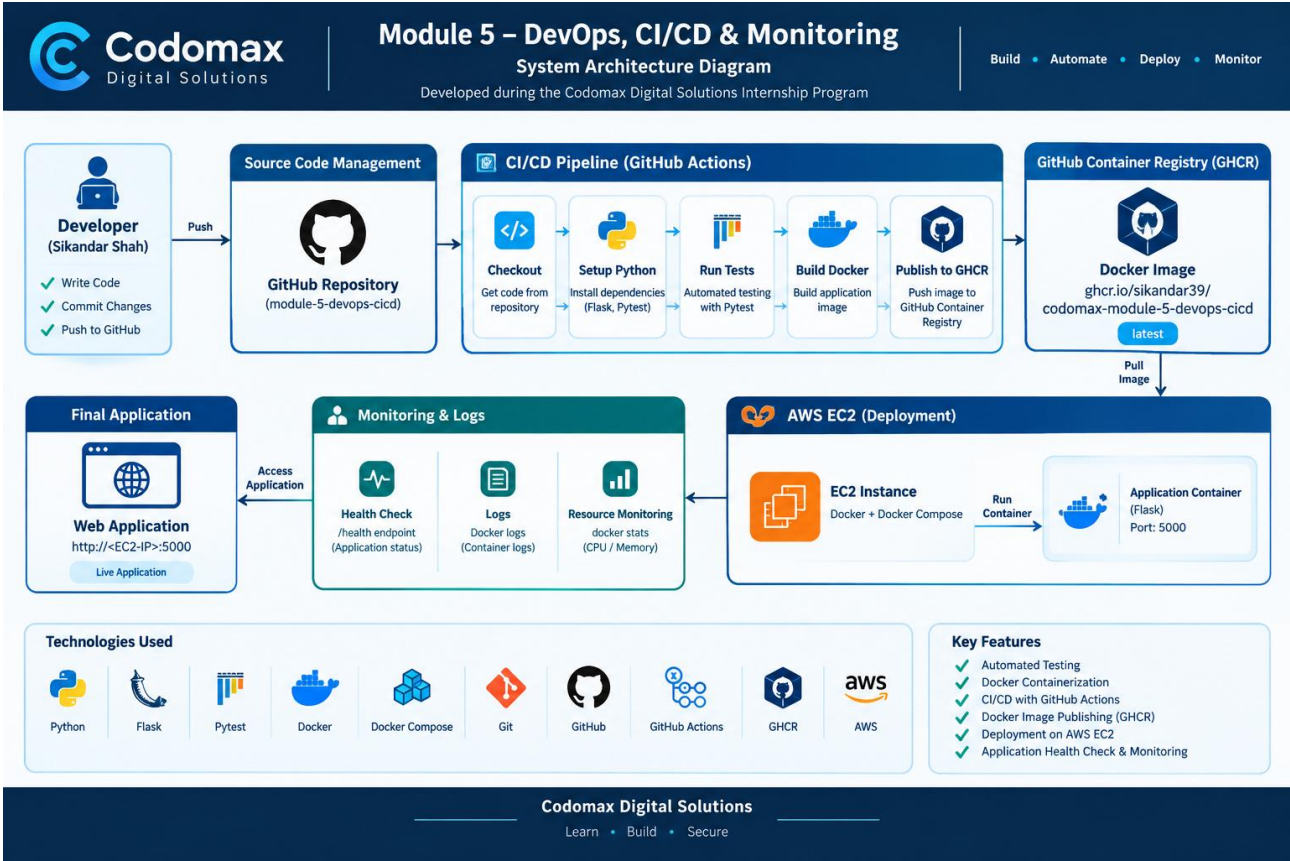


CODOMAX DIGITAL SOLUTIONS

MODULE 5

DevOps, CI/CD & Monitoring

Developer	Sikandar Shah
Program	Codomax Digital Solutions Internship
Role	DevOps, CI/CD & Monitoring



System Architecture Overview

Executive Summary

This report documents Module 5 — DevOps, CI/CD & Monitoring, completed during the Codomax Digital Solutions internship program. A Python Flask web application is used to demonstrate source control, automated testing, Docker containerization, CI/CD automation, container image publishing, deployment concepts, and basic monitoring.

The report provides a structured technical record of the implementation. Practical execution is supported by the project's screenshot evidence and GitHub workflow history.

Document Control

Document	Module 5 — DevOps, CI/CD & Monitoring
Organization	Codomax Digital Solutions
Developer	Sikandar Shah
Document Type	Technical Internship Project Report
Year	2026

Contents

- 1. Project Overview
- 2. Objectives
- 3. Technology Stack
- 4. Architecture & Workflow
- 5. Application Implementation
- 6. Automated Testing
- 7. Docker & Docker Compose
- 8. GitHub Actions CI/CD
- 9. GitHub Container Registry
- 10. Deployment & Monitoring
- 11. Security Practices
- 12. Project Structure
- 13. Evidence & Verification
- 14. Learning Outcomes
- 15. Conclusion

1. Project Overview

The project demonstrates a practical DevOps workflow for a Flask web application. GitHub is used for source management, GitHub Actions provides automation, Docker packages the application, GHCR stores the container image, and AWS EC2 represents the cloud deployment environment.

2. Objectives

- Understand DevOps and CI/CD fundamentals.
- Use Git and GitHub for source-code management.
- Implement automated testing with Pytest.
- Containerize the Flask application using Docker.
- Use Docker Compose for container management.
- Automate testing and image publishing with GitHub Actions.
- Publish a Docker image to GHCR.
- Apply health checks, logs, and basic monitoring.

3. Technology Stack

Technology	Role	Implementation
Python / Flask	Application	Web application and health endpoint
Pytest	Testing	Automated application tests
Docker	Containerization	Image and container runtime
Docker Compose	Orchestration	Service configuration and management
Git / GitHub	Source control	Repository and version history
GitHub Actions	CI/CD	Automated test, build and publish
GHCR	Registry	Container image storage
AWS EC2	Cloud	Deployment environment

4. Architecture & Workflow

GitHub → Build → Test → Docker → Publish → Deploy → Monitor

Code changes are pushed to GitHub. The CI/CD workflow validates the application, builds a portable Docker image, and publishes the image to GHCR. The published artifact can then be used for cloud deployment and operational monitoring.

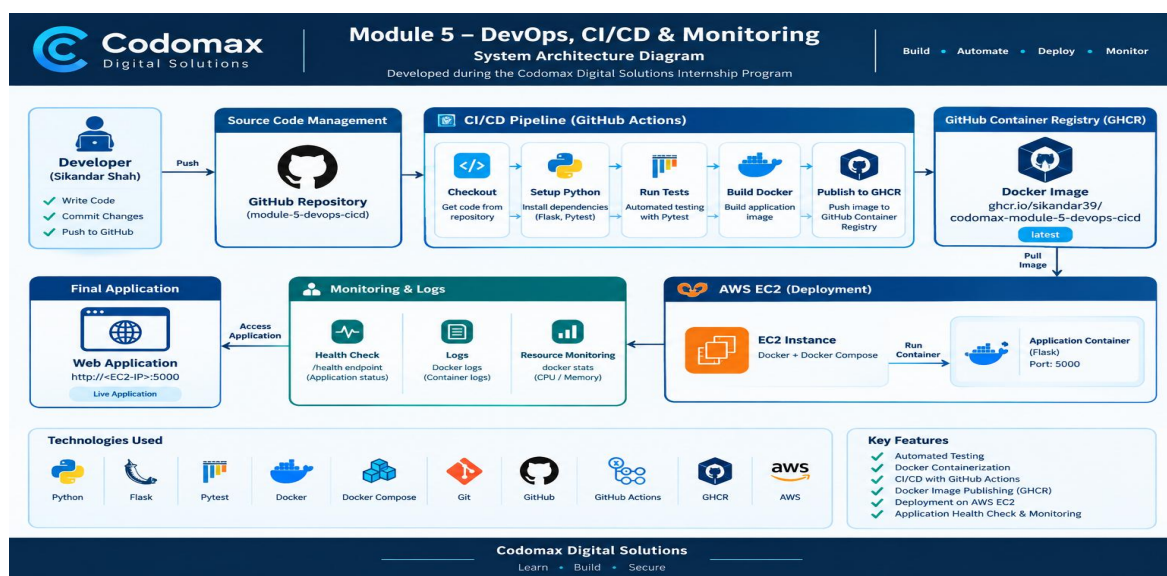


Figure 1. Module 5 DevOps, CI/CD and Monitoring Architecture

5. Application Implementation

The workload is a lightweight Flask web application. The root route provides the project interface, while the health endpoint provides a structured response for operational verification.

- / — main application page
- /health — application health endpoint

6. Automated Testing

Pytest is integrated into the project to provide repeatable automated verification. The tests validate the home page response and health endpoint. The GitHub Actions test job runs this suite before the image publishing stage.

pytest

7. Docker & Docker Compose

The Dockerfile defines a Python 3.12 Slim application image, installs dependencies, copies the project source, exposes port 5000, and starts Flask. Docker Compose provides a reusable service definition with environment variables, port mapping, and restart behavior.

- Build and run the Docker image.
- Verify the container and web application.
- Inspect container logs.
- Manage the service with Docker Compose.
- Monitor container resources.

8. GitHub Actions CI/CD

GitHub Actions automates the core CI/CD process. The workflow checks out the repository, configures Python, installs dependencies, runs Pytest, authenticates to GHCR, and builds and publishes the Docker image.

- Checkout repository
- Set up Python 3.12
- Install dependencies
- Run automated tests
- Log in to GHCR
- Build and push Docker image

9. GitHub Container Registry

The published container image uses the following lowercase GHCR path:

ghcr.io/sikandar39/codomax-module-5-devops-cicd:latest

10. Deployment & Monitoring

AWS EC2 is used as the cloud deployment environment in the project architecture. The application can run as a Docker container on EC2 using the published image. Basic operational verification uses the health endpoint, Docker logs, and Docker resource statistics.

- Application health check using /health.
- Docker container/application logs.
- Container CPU and memory statistics.
- Web application availability verification.

11. Security Practices

- Do not commit passwords, tokens, AWS credentials, or SSH private keys.
- Exclude .env and unnecessary files with .gitignore and .dockerignore.
- Use environment variables for configuration.
- Use protected GitHub Secrets for deployment credentials.
- Limit GitHub Actions permissions to required operations.

12. Project Structure

```
module-5-devops-cicd/
├── app.py
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── .dockerignore
├── .gitignore
├── .env.example
├── templates/
│   └── index.html
├── tests/
│   └── test_app.py
├── .github/
│   └── workflows/
│       └── ci-cd.yml
```

13. Evidence & Verification

The screenshot evidence provides the practical record of the implementation stages. The evidence set covers local testing, Docker, Compose, GitHub Actions, GHCR, AWS deployment, health checks, logs, and resource monitoring.

Figure 1: Project structure

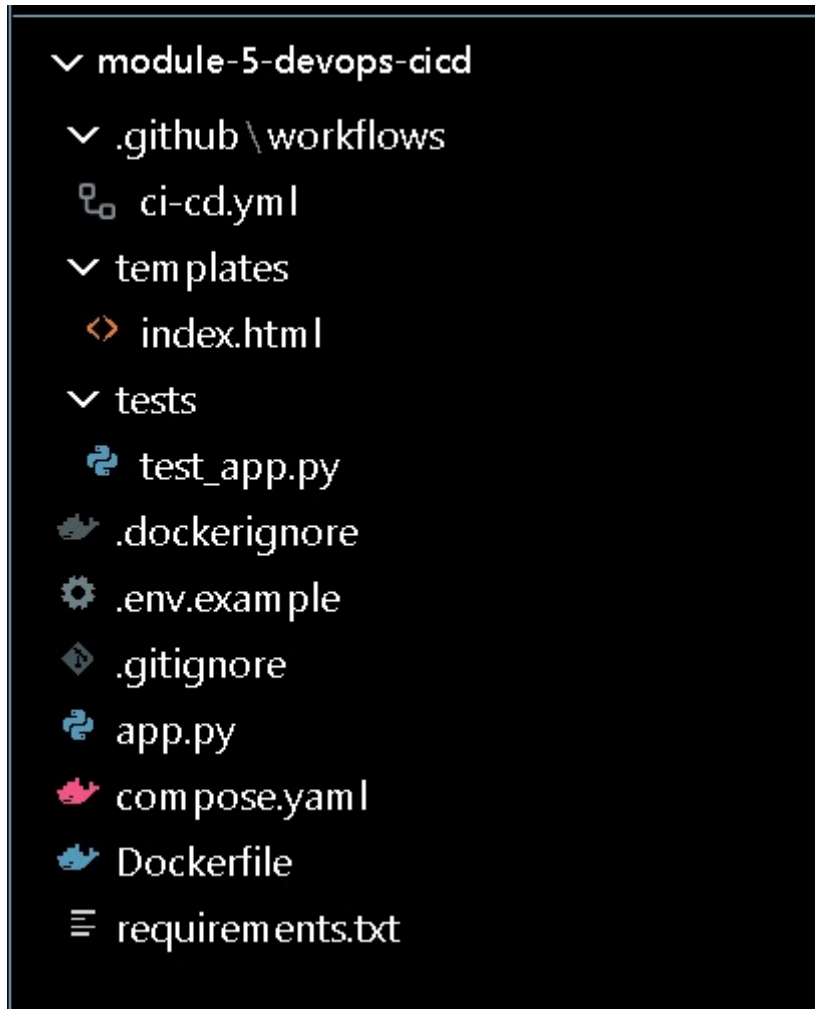


Figure 2: Local tests

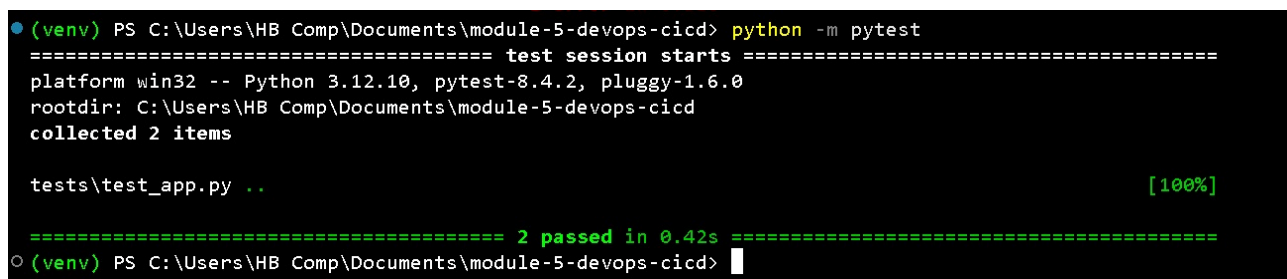


Figure 3: Application

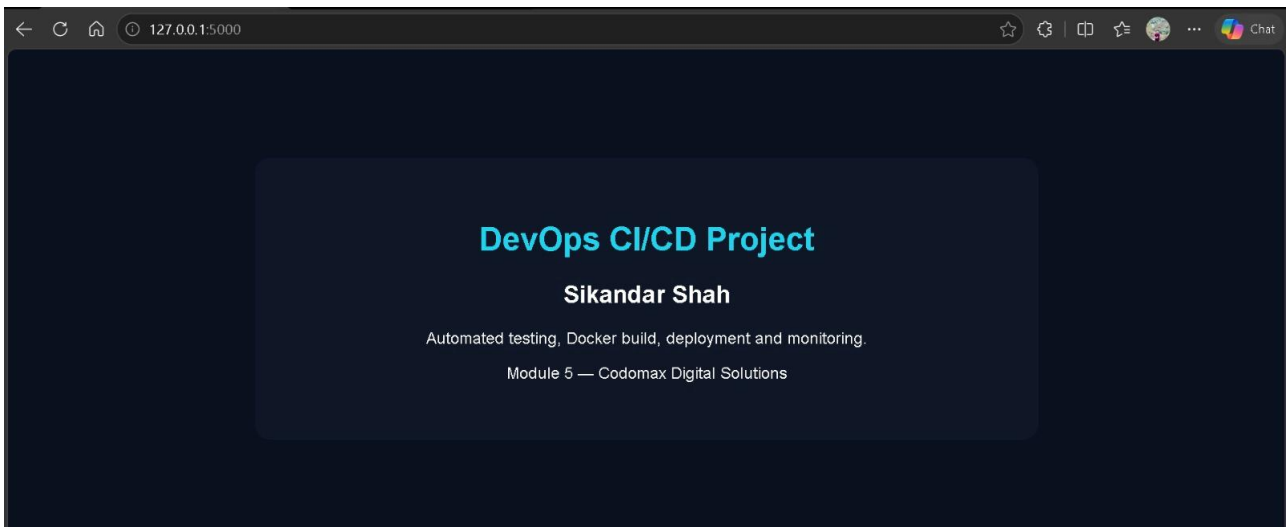


Figure 4: Health check

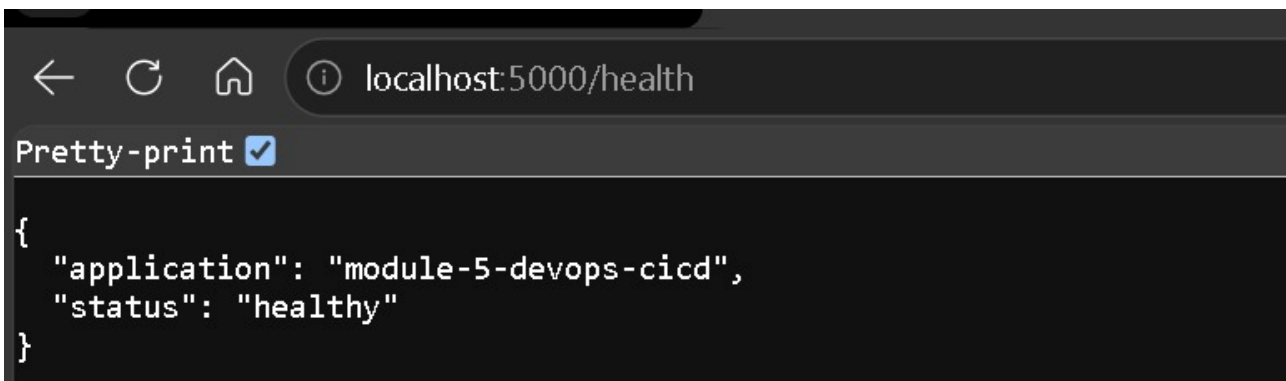


Figure 5: Docker image

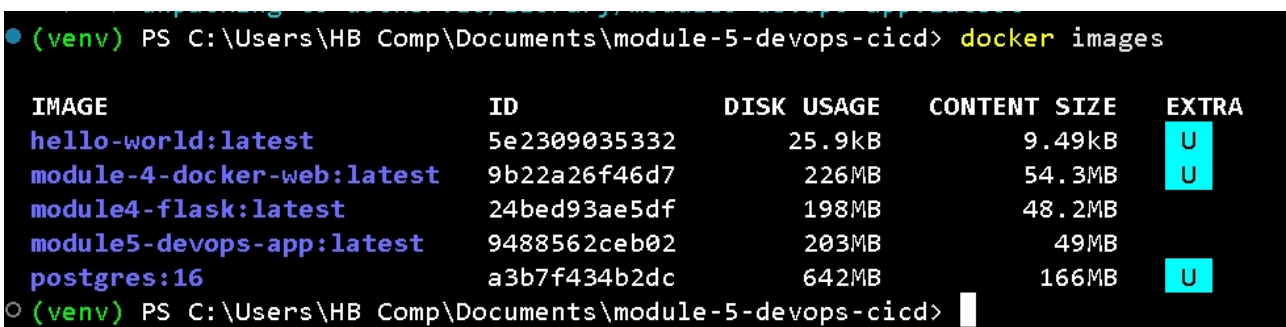


Figure 6: Container



Figure 7: Web application



Figure 8: Logs

```
(venv) PS C:\Users\HB Comp\Documents\module-5-devops-cicd> docker logs module5-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [24/Sep/2026 08:52:52] "GET / HTTP/1.1" 200 -
(venv) PS C:\Users\HB Comp\Documents\module-5-devops-cicd>
```

Figure 9: Docker Compose

```
(venv) PS C:\Users\HB Comp\Documents\module-5-devops-cicd> docker run -d --name module5-app -p 5000:5000 module
5-devops-app
06facfd8484428969932369aeb1267922112f9a493a0567c597b77f878558dd1
(venv) PS C:\Users\HB Comp\Documents\module-5-devops-cicd> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
06facfd84844   module5-devops-app                 "python app.py"         14 seconds ago Up 12 seconds  0.0.0.0:5000->5000/tcp,
[::]:5000->5000/tcp   module5-app
(venv) PS C:\Users\HB Comp\Documents\module-5-devops-cicd>
```

Figure 10: GitHub repository

```
Successfully pushed and updated refs/heads/main.
PS C:\Users\HB Comp\Documents\module-5-devops-cicd> git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (14/14), 1.95 KiB | 105.00 KiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Sikandar39/codomax-module-5-devops-cicd.git
a8fec5c..432a306 main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\HB Comp\Documents\module-5-devops-cicd>
```

Figure 11: Actions

This screenshot shows the GitHub Actions interface for a workflow named "Add CI automated testing workflow #2". The workflow is triggered by a push to the main branch. The status is "Failure" with a total duration of 11s. The workflow file is "ci-cd.yml". The job "Test Application" is shown with a duration of 8s. The annotations section indicates that the workflow completed with exit code 2 and provides information about deprecated Node.js versions and upcoming Ubuntu runner image updates.

Module 5 CI/CD

Add CI automated testing workflow #2

Triggered via push now

Sikandar39 pushed · b7201ac · main

Status: Failure

Total duration: 11s

Artifacts: -

ci-cd.yml

on: push

Test Application 8s

Annotations

1 error, 1 warning, and 1 notice

Test Application

Process completed with exit code 2.

Test Application

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/setup-python@v5. For more information see: <https://git->

Show more

Test Application

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>"

Figure 12: CI/build

This screenshot shows the GitHub Actions interface for a workflow named "Add automated Docker build #3". The workflow is triggered by a push to the main branch. The status is "Failure" with a total duration of 11s. The workflow file is "ci-cd.yml". The job "Test Application" is shown with a duration of 8s, and the job "Build Docker Image" is shown with a duration of 0s. The annotations section indicates that the workflow completed with exit code 2 and provides information about deprecated Node.js versions and upcoming Ubuntu runner image updates.

Module 5 CI/CD

Add automated Docker build #3

Triggered via push now

Sikandar39 pushed · 92851ce · main

Status: Failure

Total duration: 11s

Artifacts: -

ci-cd.yml

on: push

Test Application 8s

Build Docker Image 0s

Annotations

1 error, 1 warning, and 1 notice

Test Application

Process completed with exit code 2.

Test Application

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/setup-python@v5. For more information see: <https://git->

Show more

Test Application

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>"

Figure 13: GHCR image.

This screenshot shows the GitHub Actions interface for a workflow named "Fix GHCR image tag lowercase". The workflow is triggered by a push to the main branch. The status is "Success" with a total duration of 46s. The workflow file is "ci-cd.yml". The job "Fix GHCR image tag lowercase" is shown with a duration of 46s. The annotations section indicates that the workflow completed with exit code 0.

All workflows

Showing runs from all workflows

7 workflow runs

Workflow Event Status Branch Actor

Fix GHCR image tag lowercase

Module 5 CI/CD #7: Commit 0f54003 pushed by Sikandar39

main

3 minutes ago

46s

Figure 14: EC2 deployment

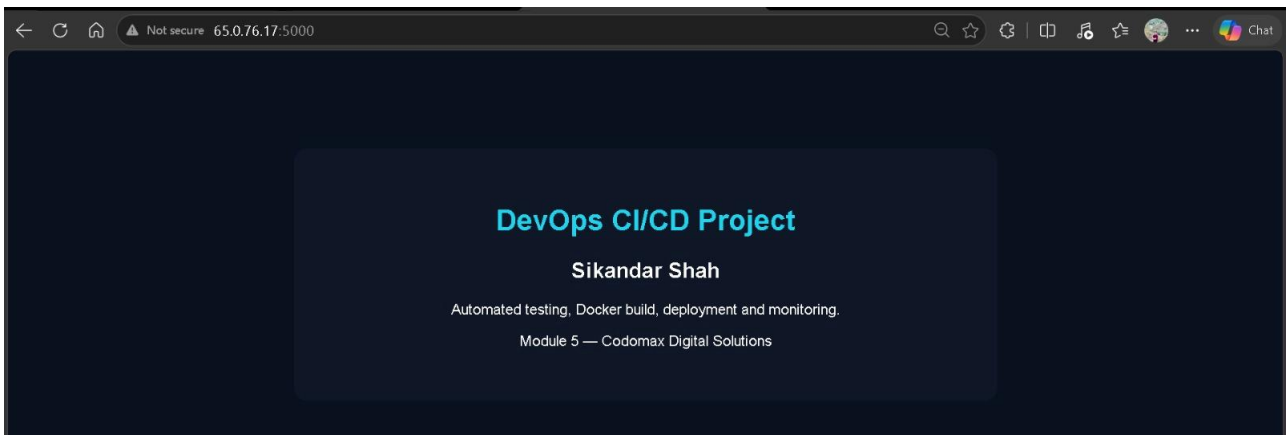


Figure 15: Deployment automation

```
ubuntu@ip-10-0-3-129:~$ nano ~/deploy.sh
ubuntu@ip-10-0-3-129:~$ ~/deploy.sh
Pulling latest Docker image...
latest: Pulling from sikandar39/codomax-module-5-devops-cicd
Digest: sha256:791555113e6476d1c3eb359b1f2ae4eefbc276ea503d0a25951575072e2dddee
Status: Image is up to date for ghcr.io/sikandar39/codomax-module-5-devops-cicd:latest
ghcr.io/sikandar39/codomax-module-5-devops-cicd:latest
Stopping old container...
module5-app
Starting new container...
b7839a206c872cec93b322e7bf411736b11e859393272986d26d632a9a87e938
Checking container...
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
b7839a206c87   ghcr.io/sikandar39/codomax-module-5-devops-cicd:latest   "python app.py"         Less than a sec
ond ago       Up Less than a second    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   module5-app
Deployment completed.
ubuntu@ip-10-0-3-129:~$
```

Figure 16: Secrets

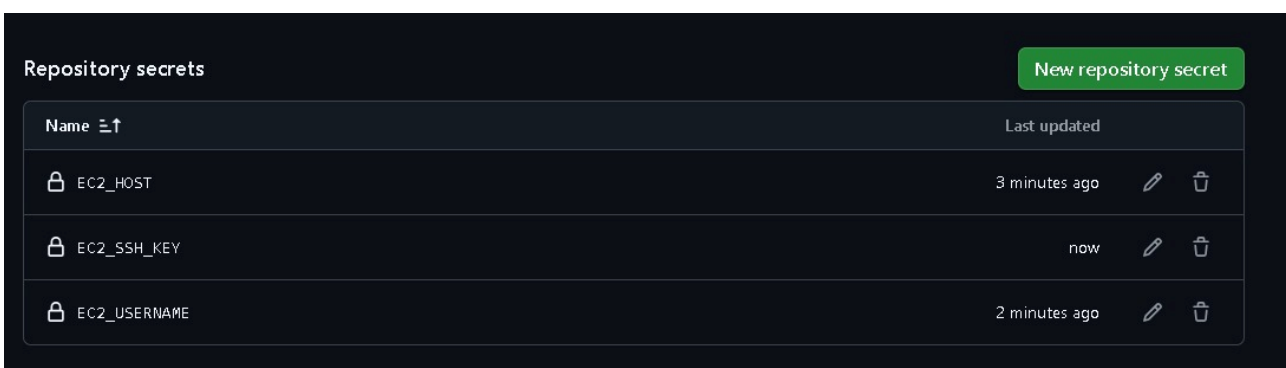


Figure 17: Environment

Environments / Configure production

Deployment protection rules

Configure reviewers, timers, and custom rules that must pass before deployments to this environment can proceed.

☐ **Required reviewers**
Specify people or teams that may approve workflow runs when they access this environment.

☐ **Wait timer**
Set an amount of time to wait before allowing deployments to proceed.

Enable custom rules with GitHub Apps Preview
[Learn about existing apps](#) or [create your own protection rules](#) so you can deploy with confidence.

☒ **Allow administrators to bypass configured protection rules**

Save protection rules

Figure 18: Pipeline

Sikandar39 / codomax-module-5-devops-cicd

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Module 5 CI/CD

Fix Docker tag to lowercase username #10

Re-run all jobs Latest #2

Summary

All jobs

Automated Tests

Build and Publish Docker Image

Deploy to AWS EC2

Run details

Usage

Workflow file

Re-run triggered 1 minute ago

Status

Total duration

Artifacts

Sikandar39 7a362a6 #10

Success

48s

1

ci-cd.yml

on: push

Automated Tests 7s

Build and Publish Docker I... 23s

Deploy to AWS EC2 8s

Figure 19: Health check

```
ubuntu@ip-10-0-3-129:~$ curl http://localhost/health
{"application":"online","database":"online","s3":"online"}
ubuntu@ip-10-0-3-129:~$
```

Figure 20: Application logs

```
ubuntu@ip-10-0-3-129:~$ docker logs module5-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
160.191.158.166 - - [24/Sep/2026 12:42:47] "GET / HTTP/1.1" 200 -
```

Figure 21: Docker resource monitoring



14. Learning Outcomes

- Practical understanding of a basic DevOps lifecycle.
- Experience integrating automated tests into CI.
- Experience building and running Docker containers.
- Experience publishing container images through GHCR.
- Understanding of deployment workflow and basic observability.
- Improved awareness of secure configuration and credential handling.

15. Conclusion

Module 5 provided practical experience in connecting software development with automation, testing, containerization, publishing, deployment, and monitoring. The resulting workflow demonstrates how a Flask application can move from source code to a reproducible container artifact through GitHub-based automation.

Project Reference

GitHub Repository:

<https://github.com/Sikandar39/codomax-module-5-devops-cicd>

END OF REPORT

Codomax Digital Solutions • Module 5 • Sikandar Shah